Individual Reflection

CSC 481/581 - Game Engine Foundations - Fall 2026

Group 4: Sai Sumedh Kaveti

Milestone 2 - Time and Networking Foundations
Project 2 - Individual Reflection

Individual Game Name: "Cave Ninja" GitHub Game
Repository:
https://github.com/CodeRockerr/CSC481-581-Group4/tree/main/Games/skaveti

(A) My Role

I created the engine's timelines (Section 1) and the multithreaded client loop (Section 3). Yeva created the server and network client, while Adit created the message format, peer-to-peer mode and loop pacing. I then networked my game, Cave Ninja, so that it displays all five sections in one place (Figures 1-10).

(B) Timelines: Why I Accumulate Time

Lecture 06 computes local time as (now - start - paused) / tic; this doesn't work if we ever change scale, since we apply the new scale factor to the entire elapsed time resulting in the player teleporting. Instead, we accumulate (anchor now - anchor last) x scale on each local timeline for each tick, and pause, unpause and setScale first accumulate the time up to the new moment; a change only affects the future, so the ninja never teleports when you press 1 or 3 (Figure 3).

Each entity has a timelineId rather than being paused directly; the platform and wisp are on WorldTime; the ninja is on PlayerTime. The button P pauses only PlayerTime, so the ninja freezes mid-air while the world continues to move (Figure 2), without any extra game code. Each timeline has a mutex and a child always locks itself before its anchor, so the threads cannot deadlock.

(C) The Multithreaded Loop

The two persistent workers based on ThreadExample.cpp [mutex, condition variable, busy flag] pattern construct each frame: where one advances PlayerTime and the other advances WorldTime and applies network snapshots. We start these threads once and reuse them to save the cost of starting new threads each frame. The hardest part was figuring out what may be accessed by workers:

1. Only the main thread calls SDL. It draws a copy of the last finished frame; thus, it can't read partially updated positions.

2. Each entity has its own mutex. Std::mutexcannot be copied, so I've put it in a copyable EntityMutex

3. Workers never create entities. New remote players are created on main thread; thus, the list is never modified while the worker iterates over it.

(D) Cave Ninja: Using the Engine

In client mode the platform is obtained from the server (Figure 4); in peer-to-peer mode each peer computes it from a common start time (Figure 6). The wisp is never transmitted over the network: its position is calculated from the platform's, so that every window displays it in the same place for free. With clients running at 0.5x, 1x and 2x, loops ran at approximately 29, 58 and 114 Hz and still agreed on the platform, since WorldTime remains at 1.0 and slower client takes larger steps (Figures 5 and 10). The server log shows each client joining on its own connection and thread (Figure 8).

(E) What Did Not Work at First

The moving platform slid into the ninja from the side and he teleported on top of it. My landing check only asks if his feet overlap the platform. Once the platform is moving then overlapping is no longer the same as landing so now he'll land only if his feet are above the platform on the previous frame (Figure 9). I also made a pause test that expected no time to have elapsed after pausing, but the first step correctly reports the time before the pause was called, so the test was wrong, not the timeline. Finally, the worker frame counters had read 0 by the end of the loop, because I reset the workers too soon, which taught me to reason about shutdown order just as I would start-up.

The important lesson is that rules that apply to single thread and objects static across time (read an entity at any time, consider overlapping to be a landing) can stop applying when time, multiple threads and the network get involved. Most of my bugs came from an assumption like that, rather than an outright typo.

(F) Limitations and What it Enables

There are four known limitations in the current version. One, there is no moving average on the frame time, so a single slow frame produces a single huge time step instead of being smoothed out by others. Second, remote players are not interpolated, so on the 0.5x client, the other ninjas appear a few pixels behind where they should be. Third, players who have left the game are not removed from the other screens. And, the moving platform has no collision on its sides, so it moves straight through the ninja instead of pushing him out of the way. The design, however, makes all of these easy to fix. Each network update already includes a timeline tic, so interpolating between two tics is a logical next step. Time, like many other things in this design, is attached to timelines rather than entities, so features such as per-object slow motion or replays only require a new timeline. Similarly, because workers only ever access entities through their mutexes, more workers can be added with no changes to the game code.

(G) Attribution

The worker thread design was inspired by the course's ThreadExample.cpp, and the timeline design was inspired by the Lecture 06 material on time management. Networking used libzmq 4.3.5, and rendering used SDL3 with stb_image for loading textures.

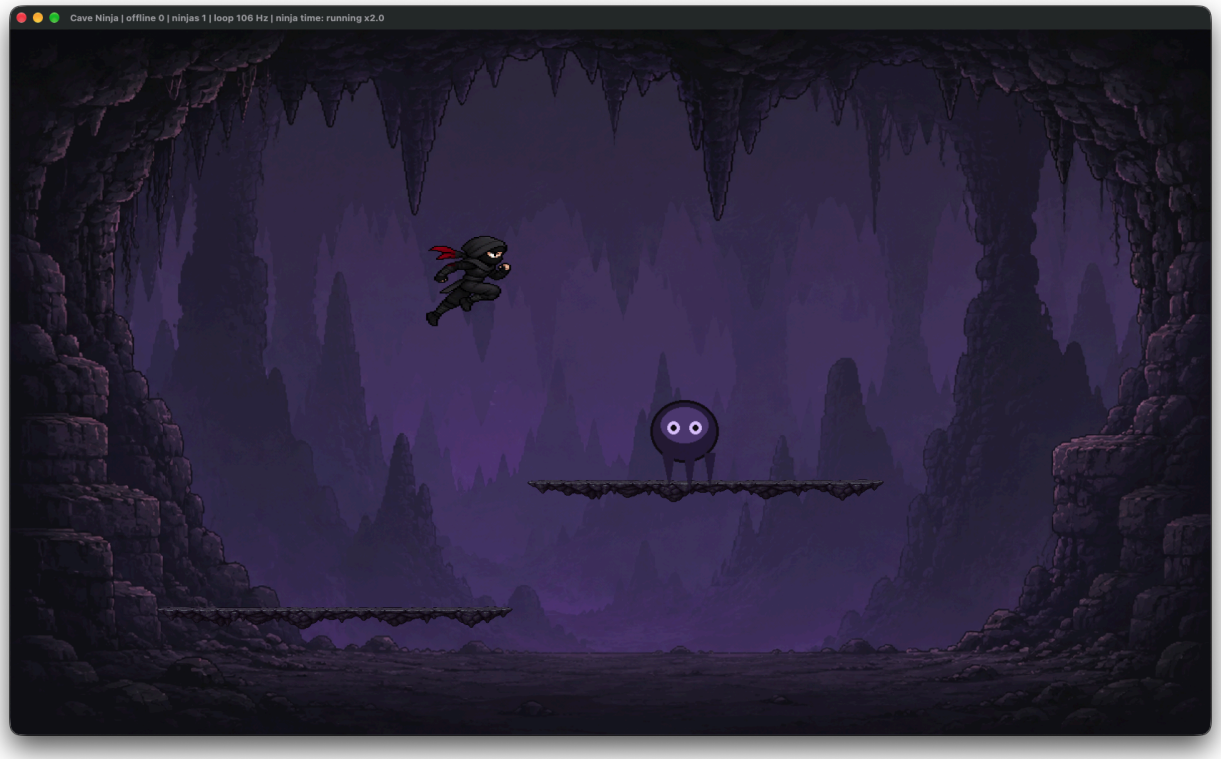
Appendix - Screenshots



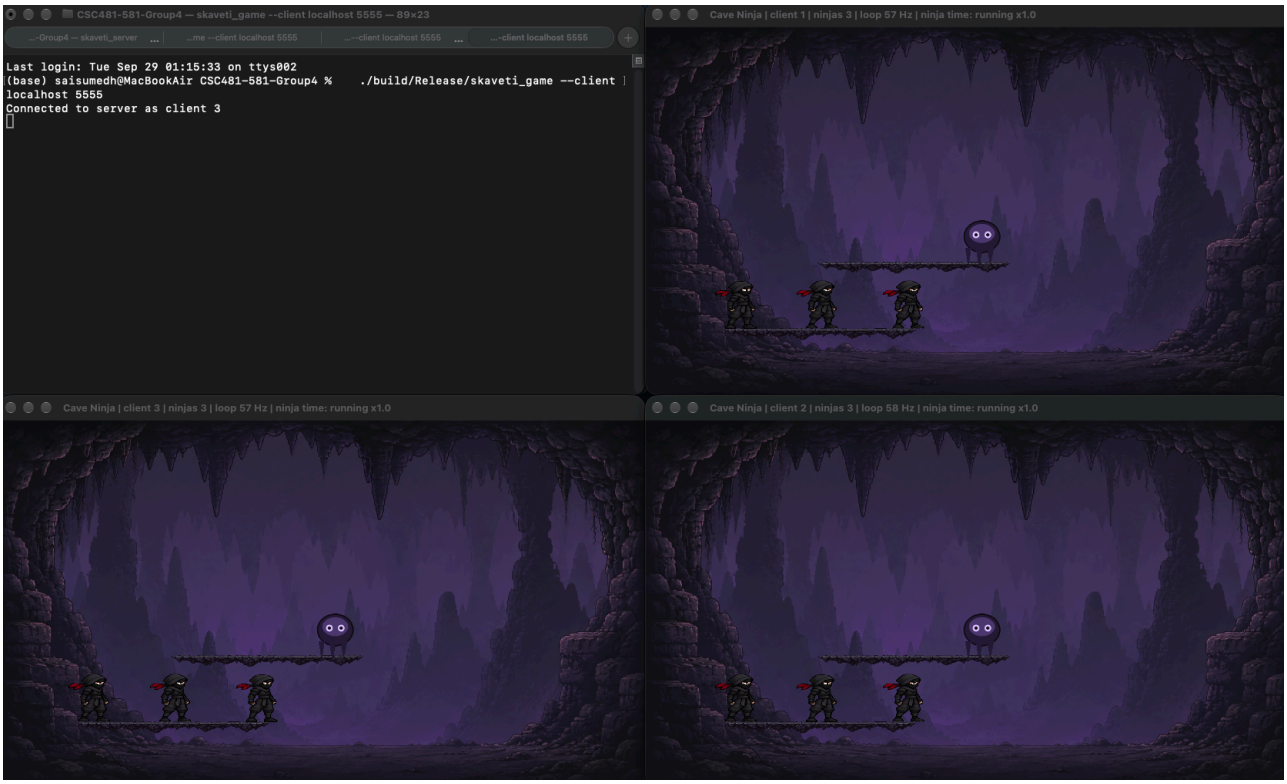
(fig. 1 - Offline play, ninja riding the moving platform)



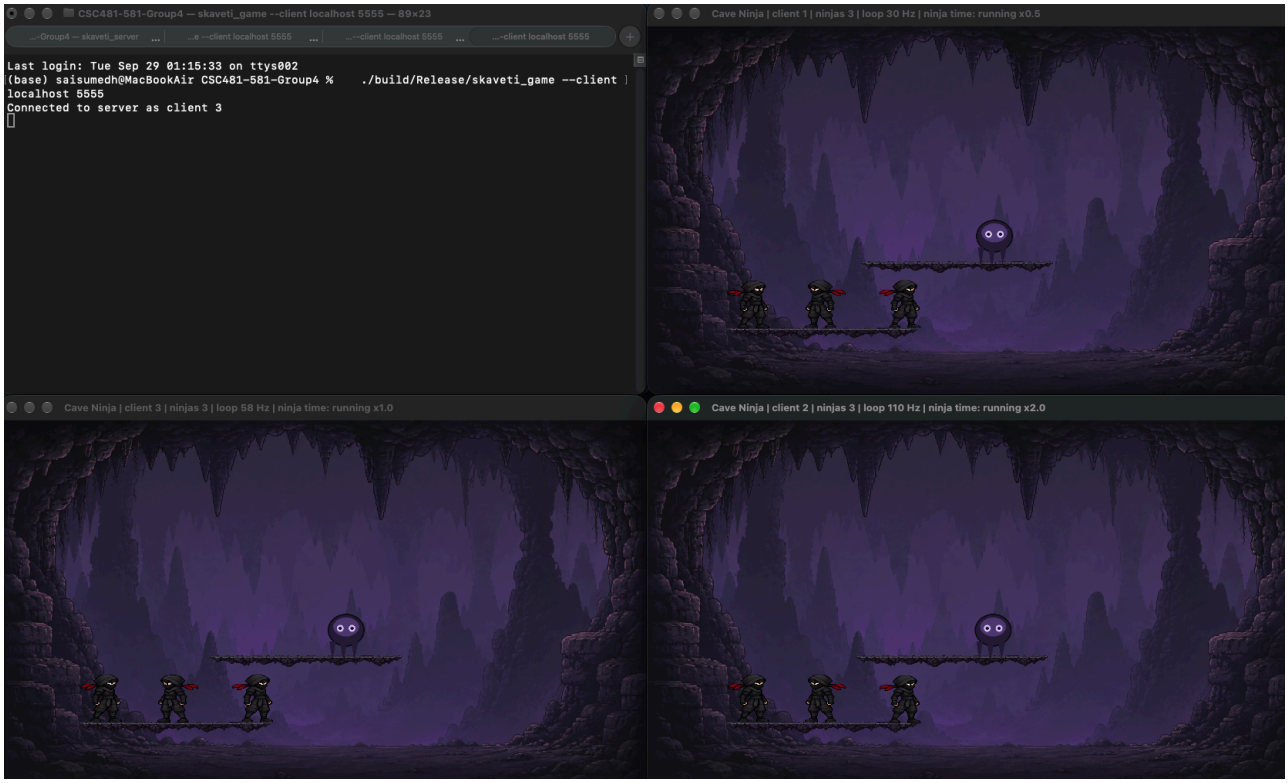
(fig. 2 Ninja paused with P while the platform and wisp kept moving)



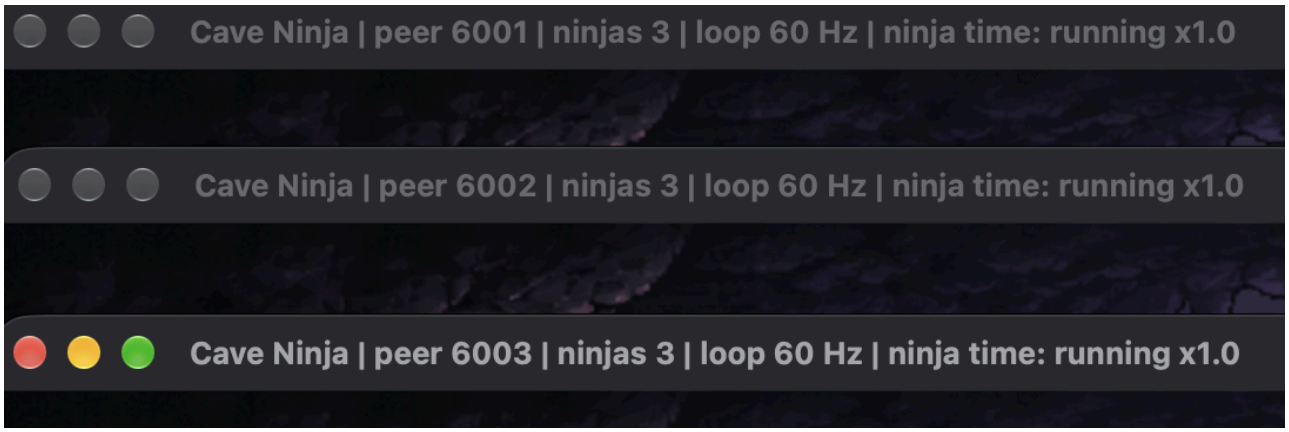
(fig. 3 Ninja at 2x speed, scale shown in the title bar)



(fig. 4 Server + 3 clients: same platform and 3 ninjas in every window)



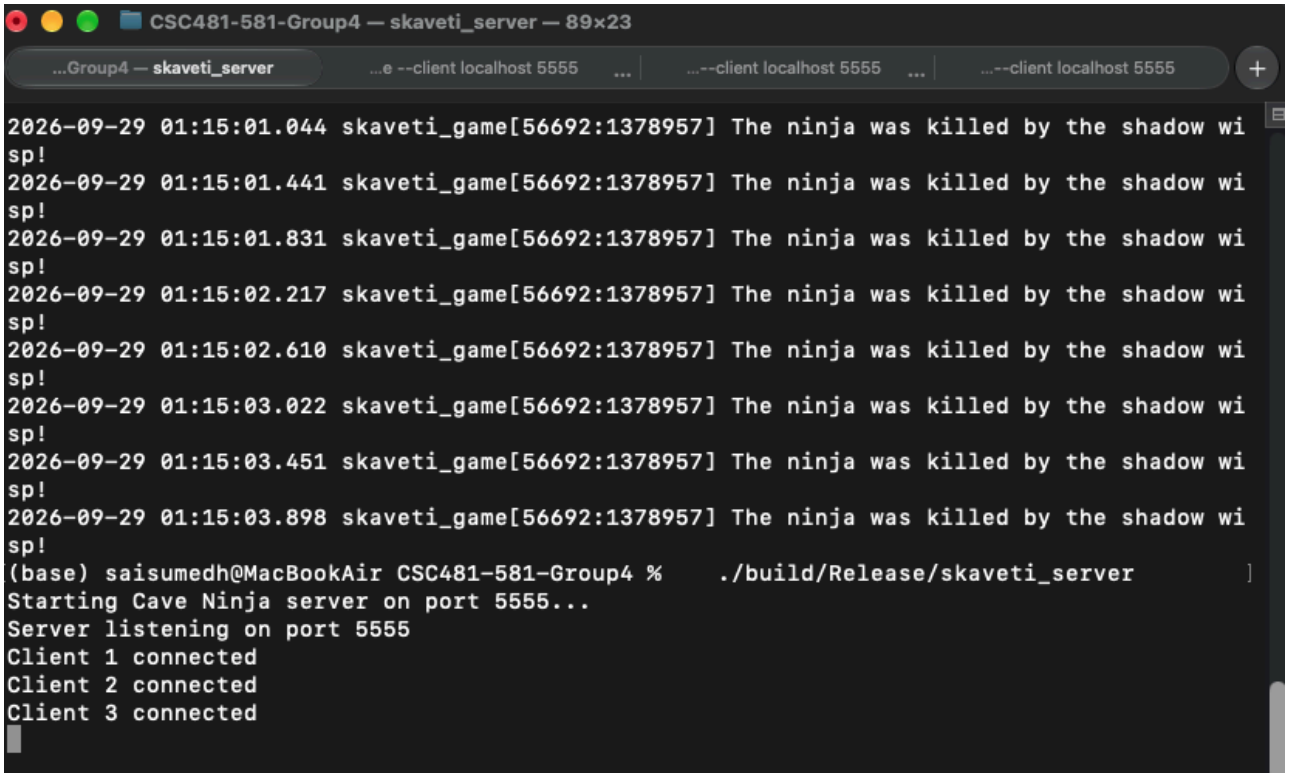
(fig. 5 Clients at 0.5x, 1x and 2x: different loop rates, same world)



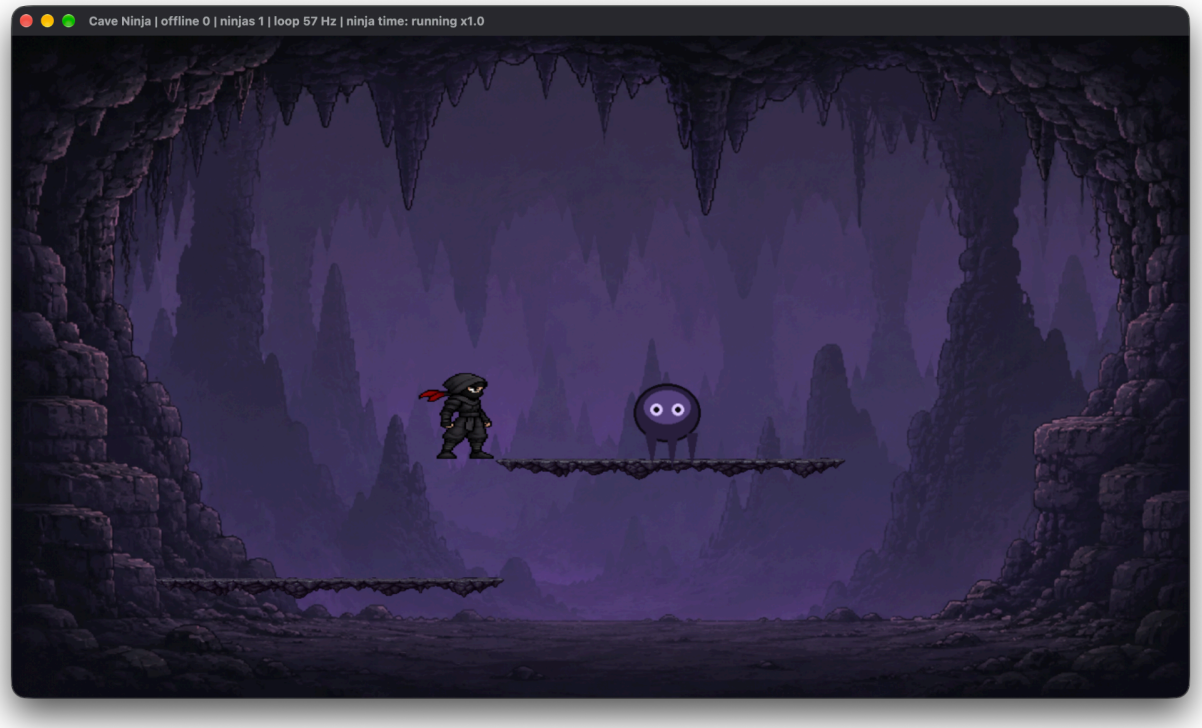
(fig. 6 Peer-to-peer with 3 peers and no server running)



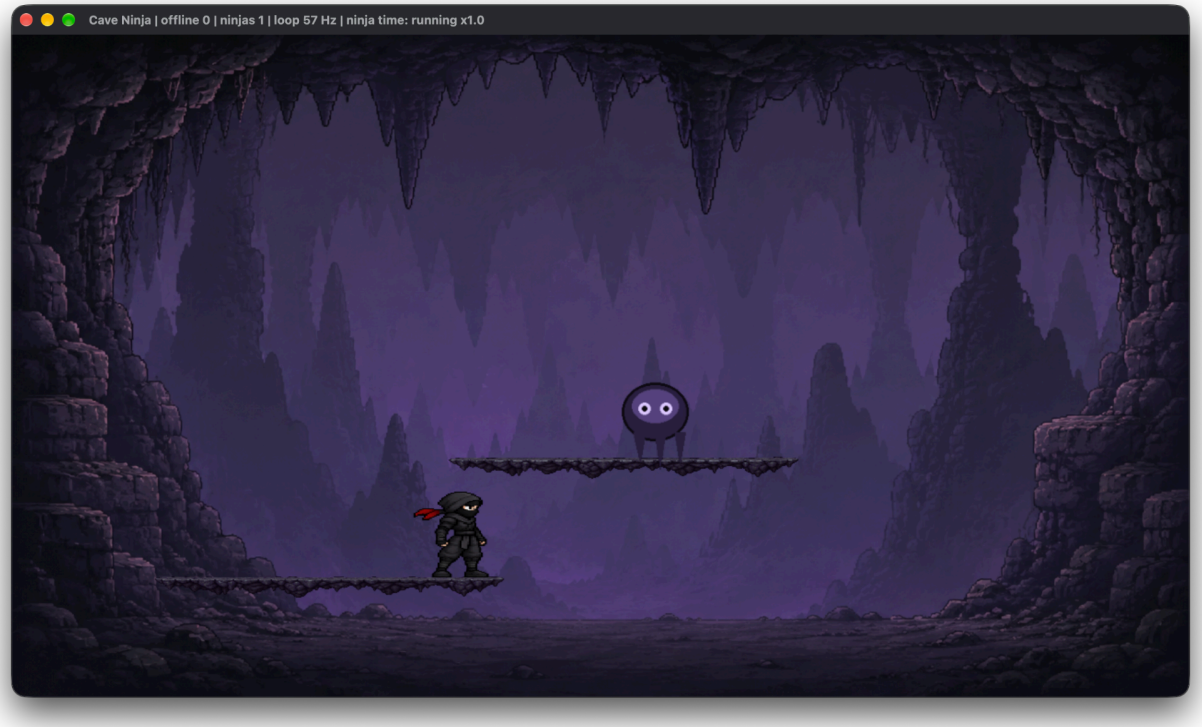
(fig. 7 Two peers still running after the third is closed)



(fig. 8 Server terminal showing three clients joining, each on its own connection)

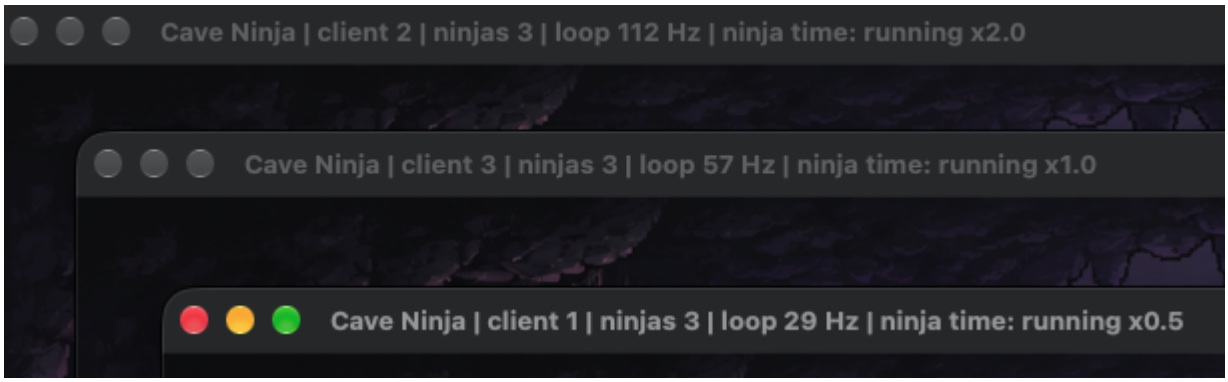


Before



After

(fig. 9 Moving platform bug: before the fix the ninja snaps on top of the platform, after the fix he stays on the stationary platform)



(fig. 10 Title bars during the async test: loop rate and ninja time scale for the 0.5x, 1x and 2x clients)